

Inheritance

For Loops

Python

Java

Iterating Over Indices

```
fruits = ["apple", "banana", "cherry"]

for i in range(len(fruits)):
    print(f'Index {i}: {fruits[i]}')
```

```
String[] fruits = {"apple", "banana", "cherry"};

for (int i = 0; i < fruits.length; i++) {
    System.out.printf("Index %d: %s\n", i, fruits[i]);
}
```

Iterating Over Items Directly

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(f'Item: {fruit}')
```

```
String[] fruits = {"apple", "banana", "cherry"};

for (String fruit : fruits) {
    System.out.printf("Item: %s\n", fruit);
}
```

Printf

```
String[] fruits = {"apple", "banana", "cherry"};
```

```
for (int i = 0; i < fruits.length; i++) {
```

```
    System.out.printf("Index %d: %s%n", i, fruits[i]);
```

```
//    System.out.println("Index " + i + ": " + fruits[i]);
```

```
}
```

Output:

```
Index 0: apple
Index 1: banana
Index 2: cherry
```

Child Constructor – Two Cases

Case 1

(Parent has a Default Constructor)



```
public class Person {  
    private String name;  
    private int age;  
  
    public Person() { // default constructor  
        System.out.println("Person created!");  
    }  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

**Child doesn't need to provide a constructor
(JAVA CALLS THE DEFAULT CONSTRUCTOR OF THE PARENT)**



```
public class Student extends Person{  
  
    private String major;  
  
}
```

Case 2

(No Default Constructor at Parent)

```
public class Person {  
    private String name;  
    private int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

**The child has to provide an explicit
constructor that calls super**

```
public class Student extends Person{  
  
    private String major;  
  
    public Student(String name, int age, String major) {  
        super(name, age);  
        this.major = major;  
    }  
}
```

Setting Attribute Values For Subclass Objects

With No Constructor

```
public class Student extends Person{  
    private String major;  
}
```



```
Student s = new Student();  
s.setName("Ahmed");  
s.setAge(18);  
s.setMajor("IS");
```

With Explicit Constructor

```
public class Student extends Person{  
    private String major;  
  
    public Student(String name, int age, String major) {  
        super(name, age);  
        this.major = major;  
    }  
}
```

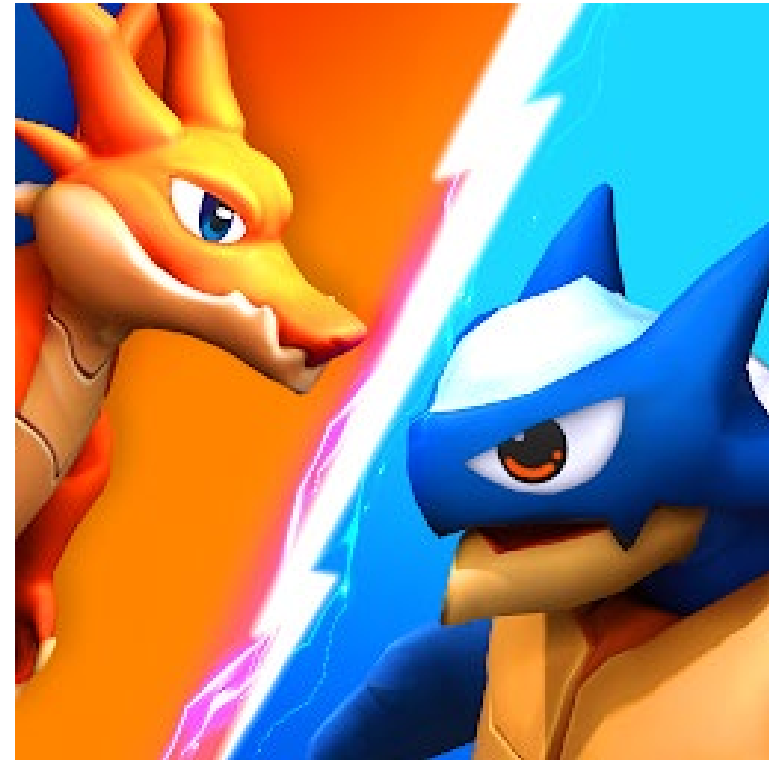


```
Student s = new Student("Ahmed", 18, "IS");
```

Monster Battle Simulation

 Monster

-  name : String
-  hp : int
-  dmg : int
-  Monster(String, int, int)
-  getName() : String
-  setName(String) : void
-  getHp() : int
-  setHp(int) : void
-  getDmg() : int
-  setDmg(int) : void
-  toString() : String
-   main(String[]) : void



Code Tracing Practice

```
public class Animal {
    protected String type;

    public Animal(String type) {
        this.type = type;
        System.out.println("Animal constructor: " + type);
    }

    public void speak() {
        System.out.println("Generic sound");
    }

    public String toString() {
        return "Animal: " + type;
    }
}

public class Cat extends Animal {
    private int lives;

    public Cat(String type, int lives) {
        super(type);
        this.lives = lives;
        System.out.println("Cat constructor: lives = " + lives);
    }

    @Override
    public void speak() {
        System.out.println(type + " says Meow!");
    }

    public String toString() {
        return "Cat (" + type + ", " + lives + " lives)";
    }

    public static void main(String[] args) {
        Animal a = new Cat("Tabby", 9);
        a.speak();
        System.out.println(a);
    }
}
```